Day 110: A Production Boosting Library — XGBoost Against the From-Scratch Version

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 16, Day 6 | Complete study guide

Watch: XGBoost Part 1 (of 4): Regression (StatQuest) | XGBoost in Python from Start to Finish (StatQuest)

Script: `day110_xgboost_in_practice.py` · every number in this guide is copied by program from `day110_run_output.txt`, the real output of that script.

Is XGBoost the same algorithm as Day 109? Nearly, and the difference can be written down. With its Newton split rule from scratch, all 500 of 500 trees come out identical on breast cancer, to within 0.000001 log-odds. What the library really adds is engineering: at 30,000 rows it is about 67 times faster on one thread (16.64 s against 0.25 s). Not everything people expect held up: its missing-value handling did not beat a median fill, tuning helped on one dataset and not the other, and on digits a from-scratch forest (0.9712) beat it (0.9576).

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

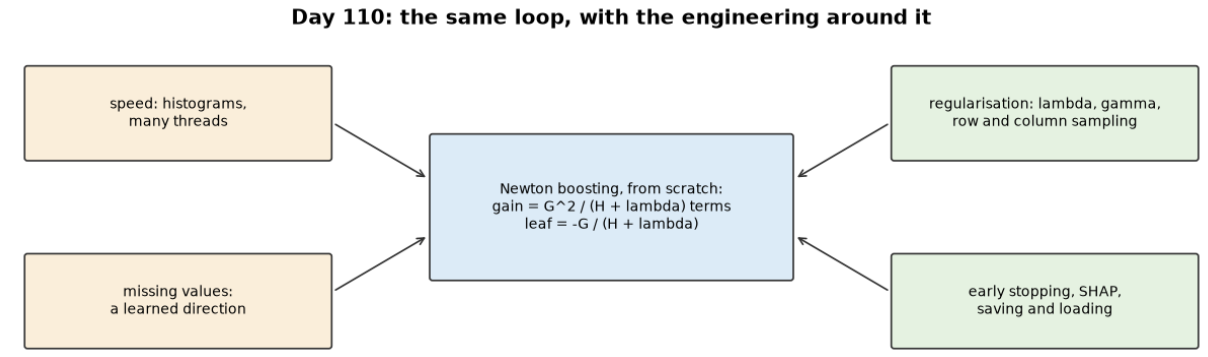
XGBoost is a widely deployed gradient-boosting library. Its central idea is **Newton boosting**: trees are chosen and their leaves set using both the gradient and the curvature of the loss, with a penalty λ on leaf values.

Day 109 used the curvature only for leaf values, and chose splits on the gradient alone. XGBoost uses it for both.

REAL-WORLD SCENARIO

A home cook and a restaurant may use the same recipe. The restaurant's advantage is its kitchen: speed, repeatability and handling whatever comes through the door.

SYSTEM DIAGRAM



The same boosting loop in the middle, and what a production library builds around it.

Today's part for Day 111

`day110_exercise.py` is Newton boosting from scratch and a small wrapper around the library, checked by hand and against XGBoost. Its real output:

ACTUAL OUTPUT

```
from F = 0, lambda = 1: split at 2.5; right leaf -G / (H + lambda) = 20 / 3 = 6.6667
gain by hand: 0.5 * (0 + 400 / 3 - 400 / 5) = 26.6667; newton_split: 26.6667
leaves: [-0.0, -0.0, 6.6667, 6.6667]
30 rounds on 200 random rows, log loss: largest gap to XGBoost 0.000001
```

Data: Diabetes, breast cancer, Titanic and digits, as on Days 105 to 109, plus one synthetic table used for timing only. XGBoost 3.4.0 was already installed; nothing new was downloaded.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. Is it the same algorithm?

DEFINITION — start here, no prior knowledge assumed

Per row: g is the gradient of the loss and h its curvature. G and H are their sums over a group of rows. A leaf's best value is –G / (H + λ), and a split is scored by how much the two children improve on the parent.

To test this, Newton boosting is written from scratch and compared with XGBoost tree by tree, with its `exact` method and the same settings. A test of 'same accuracy' would be weak; 'same trees' is strong.

REAL-WORLD SCENARIO

Rebuilding a watch from its drawings: if every part fits and it keeps the same time, you have understood it.

MATHEMATICS

$$leaf = -G / (H + \lambda)$$

$$gain = \tfrac{1}{2} \left[G_L^2/(H_L+\lambda) + G_R^2/(H_R+\lambda) - G^2/(H+\lambda) \right]$$

squared error: $g = F - y,\ h = 1.$ *Log loss:* $g = p - y,\ h = p(1 - p)$

IMPLEMENTATION

```
def newton_split(X, g, h, lam):
    n, best = len(g), None
    G, H = g.sum(), h.sum(); parent = G ** 2 / (H + lam)
    for f in range(X.shape[1]):
        o = np.argsort(X[:, f], kind="stable"); xs = X[o, f]
        GL = np.cumsum(g[o])[:-1]; HL = np.cumsum(h[o])[:-1]
        ok = xs[1:] != xs[:-1]
        if not ok.any():
            continue
        gain = np.where(ok, 0.5 * (GL ** 2 / (HL + lam) + (G - GL) ** 2 / (H - HL + lam) -
            parent), -np.inf)
        i = int(np.argmax(gain))
        if best is None or gain[i] > best[2] + 1e-12:
            best = (f, (xs[i] + xs[i + 1]) / 2, float(gain[i]))
    return best if best and best[2] > 1e-12 else None
```

ACTUAL OUTPUT

```
xgboost 3.4.0; tree_method exact; no minimum child weight; 100 rounds, rate 0.1, depth 3, the same starting
    value; 5 splits
our predictions use XGBoost's convention: 32-bit values, and a row exactly on a threshold goes right
problem                      lambda  same trees  train: mean gap  test: mean gap   largest  pred std
Diabetes (squared error)        0.0     466 of 500     0.467458        0.568782   25.7870    56.077
Diabetes (squared error)        1.0     492 of 500     0.000023        0.070697   10.6103    55.064
Diabetes (squared error)       10.0     490 of 500     0.000021        0.045441    9.4587    54.910
Breast cancer (log loss)        0.0     404 of 500     0.000315        0.000333    0.0034     7.703
Breast cancer (log loss)        1.0     500 of 500     0.000001        0.000001    0.0000     5.687
Breast cancer (log loss)       10.0     500 of 500     0.000001        0.000001    0.0000     4.014
the first tree to differ, in the 15 of 30 runs where any did: at the first differing question the two gains
    differ by 1.8e-06 (median), 4.1e-03 (largest), relative
Day 109's rule on log loss chooses splits on the gradient alone; its mean gap to the Newton rule is 1.3181
Actual output from this lesson's run.
```

What it means

On breast cancer with $\lambda = 1$ or 10, every tree is identical and predictions agree to 0.000001. That is the algorithm, reproduced from its formula. The first attempt looked worse, and the reason is a useful detail: XGBoost sends a row exactly on a threshold to the right and stores values in 32 bits, and both had to be matched here. The remaining differences are near-ties: in the 15 of 30 runs where any tree differed, the two gains at the first differing question agree to 1.8e-06 (median) and 4.1e-03 at worst. One tied split then changes all later trees, which is why a gap as large as 25.7870 is possible on Diabetes at $\lambda = 0$, against a prediction spread of 56.077. Day 109's rule is genuinely different: its mean gap is 1.3181.

WHY CHOOSE THIS?

Shows exactly what the library computes, with nothing taken on trust.

WHY NOT THIS? (TRADEOFFS)

Only for the exact method; the fast histogram method used later makes approximations.

2. Speed

DEFINITION — start here, no prior knowledge assumed

The **exact** method sorts each feature, as Day 109 does. The **histogram** method first sorts values into about 256 bins and then only ever considers bin edges, so its cost grows with the bins, not with the rows at each split.

REAL-WORLD SCENARIO

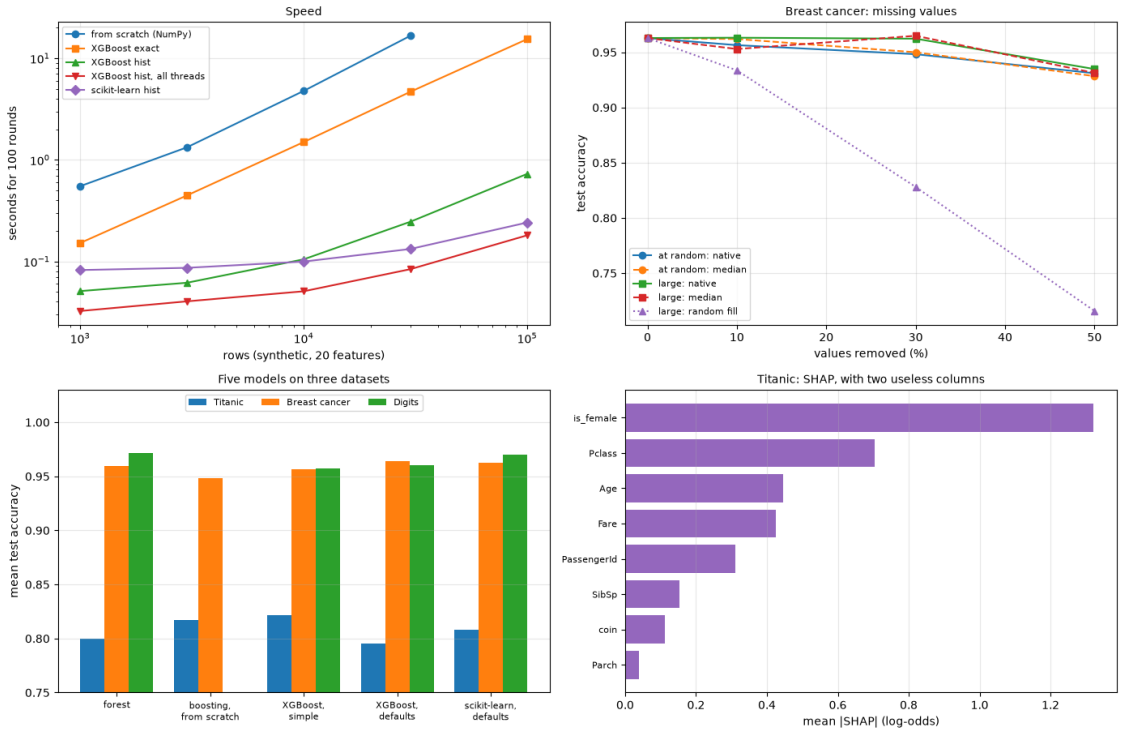
Checking a ledger line by line, against adding up the subtotals for each page.

SPECIFICATION

Data	A synthetic table for timing only: 20 features, up to 100,000 rows.
Model	100 rounds, rate 0.1, depth 3; median of 5 fits.
Threads	One, except the 'all threads' column; scikit-learn was left at its default, which uses them all.
Note	Timings here vary between runs, so compare within this table. The first run had a spike from other work on this computer, and was repeated.

ACTUAL OUTPUT

```
synthetic table for timing only: 20 features, 100 rounds, rate 0.1, depth 3; median of 5 fits (1 fit when one
    takes over 20 s); timed within this one run
    rows  from scratch  xgb exact  xgb hist  xgb hist, all threads  sklearn hist
    1000          0.55       0.15      0.05                  0.03          0.08
    3000          1.33       0.45      0.06                  0.04          0.09
   10000          4.77       1.50      0.10                  0.05          0.10
   30000         16.64       4.68      0.25                  0.08          0.13
  100000       not run      15.44      0.73                  0.18          0.24
16 logical processors; scikit-learn's threads were left at their default, so that column uses them all
Actual output from this lesson's run.
```



Top left: speed. Top right: missing values on breast cancer. Bottom left: five models on three datasets. Bottom right: SHAP on Titanic.

What it means

From scratch takes 16.64 s at 30,000 rows. XGBoost's exact method, the same algorithm compiled, takes 4.68 s, and histograms 0.25 s on one thread and 0.08 s on all of them. At 100,000 rows, histograms (0.73 s) are about 20 times faster than exact (15.44 s). The compiled exact method is only about 3.5 times faster than NumPy: most of the speed comes from the better algorithm, not the faster language.

WHY CHOOSE THIS?

Makes tuning, cross-validation and large tables practical.

WHY NOT THIS? (TRADEOFFS)

Parallel speed-up depends on the machine; for small tables thread overhead can outweigh it.

3. Missing values

DEFINITION — start here, no prior knowledge assumed

XGBoost does not need gaps filled. At each question it learns a **default direction**: the side that gaps go to. The usual alternative is to fill each gap with the training median. The surprising test is the third column, a **random fill** from observed values, which leaves no trace that a value was ever missing.

REAL-WORLD SCENARIO

A blank on a form can itself be informative: a patient who skipped a test often skipped it for a reason.

SPECIFICATION

Titanic	177 ages really are missing.
Breast cancer	Values removed either at random or from the top of each feature, so that a gap carries information.
Model	200 rounds, rate 0.1, depth 3, 20 splits; medians from training rows only.

ACTUAL OUTPUT

```
Titanic, 177 of 891 ages really missing, 20 splits, 200 rounds:
  native handling 0.8114, median fill 0.8170, gap -0.0056 [-0.0108, +0.0000]
breast cancer, values removed from every feature, 20 splits, 200 rounds:
  'at random': any value may go. 'large': only the largest in each feature go, so the gap itself is
      information.
  'random fill' puts a random observed value of that feature in each gap, so a filled value does not stand out
  removed  at random: native   median  large: native   median  random fill  native - median   +- SE
       0%             0.9629   0.9629         0.9629   0.9629       0.9629          +0.0000  0.0000
      10%             0.9564   0.9620         0.9632   0.9529       0.9336          +0.0102  0.0037
      30%             0.9482   0.9500         0.9623   0.9649       0.8281          -0.0026  0.0026
      50%             0.9310   0.9284         0.9348   0.9313       0.7158          +0.0035  0.0049
Actual output from this lesson's run.
```

What it means

The expected win did not appear. On Titanic, median fill is slightly ahead (0.8170 against 0.8114). On breast cancer, the gap between native and median stays within noise at most rates (+0.0102 at 10%, -0.0026 at 30%). The random fill explains why: with the top half of values gone, it collapses to 0.7158 while median fill keeps 0.9313. A median fill puts hundreds of copies of one exact value into a column, and trees can isolate that value with two questions, so it works as a 'this was missing' flag. The signal survives because a tree can read the gap either way. Random fill destroys that, so it is a warning for models that cannot isolate a value, as a linear model cannot.

WHY CHOOSE THIS?

Native handling needs no decision about what to fill in, and is simple at prediction time.

WHY NOT THIS? (TRADEOFFS)

On these tables it gave no accuracy advantage over a median, which trees can read just as well.

4. Tuning

DEFINITION — start here, no prior knowledge assumed

Hyperparameters are settings chosen before training. **Random search** tries random combinations and scores each by cross-validation on the training part, with **early stopping** choosing the rounds.

REAL-WORLD SCENARIO

Tasting a sauce at twenty different seasonings before serving it, always on a spare spoonful, never on the guests.

IMPLEMENTATION

```
def sample_config(rng):
    pick = lambda v: v[rng.randint(len(v))]
    return dict(eta=pick([0.02, 0.05, 0.1, 0.3]), max_depth=pick([2, 3, 4, 6, 8]),
        reg_lambda=pick([0.1, 1, 10]), gamma=pick([0, 0.1, 1]),
                subsample=pick([0.6, 0.8, 1.0]), colsample_bytree=pick([0.5, 0.8, 1.0]),
                    min_child_weight=pick([0, 1, 5]))
```

ACTUAL OUTPUT

```
20 random settings per split, each scored by 3-fold log loss on the training part with early stopping (at most
    1,000 rounds, patience 30)
10 splits; the test part is used once
Titanic: defaults (rate 0.3, depth 6, 100 rounds) 0.8052; rate 0.1, depth 3, early stopping 0.8190; tuned
    0.8153
  tuned minus simple -0.0037 [-0.0134, +0.0063]; better on 3, worse on 6 of 10
  gap between the best and worst of the 20 settings in CV log loss: 0.0241 on average
Breast cancer: defaults (rate 0.3, depth 6, 100 rounds) 0.9678; rate 0.1, depth 3, early stopping 0.9614;
    tuned 0.9731
  tuned minus simple +0.0117 [+0.0053, +0.0193]; better on 8, worse on 0 of 10
  gap between the best and worst of the 20 settings in CV log loss: 0.0481 on average
Actual output from this lesson's run.
```

What it means

Mixed. On breast cancer tuning helps (+0.0117, better on 8 of 10 and never worse). On Titanic it does not (-0.0037). The library's defaults overfit Titanic (0.8052, against 0.8190 for a low rate and shallow trees), yet beat that simple setting on breast cancer (0.9678 against 0.9614). One caution: the simple setting gives 25% of its training rows to early stopping, and the tuned model is refitted on everything, so the comparison slightly favours tuning.

WHY CHOOSE THIS?

Can find a real gain, at modest cost with a fast library.

WHY NOT THIS? (TRADEOFFS)

On small tables the choice can fit noise; a sensible simple setting often does as well.

5. The comparison

DEFINITION — start here, no prior knowledge assumed

Five models on the same splits: Day 107's forest, Day 109's boosting, XGBoost with simple settings and early stopping, XGBoost's defaults, and scikit-learn's histogram boosting at its defaults.

REAL-WORLD SCENARIO

The same five runners on three different courses.

ACTUAL OUTPUT

```
forest: 100 trees (Day 107). Boosting: rate 0.1, depth 3, at most 500 rounds, stopped on 25% of the training
    part. Library defaults: as shipped
Titanic: 891 rows, 20 splits
  forest, from scratch      0.7996  (std 0.0208)
  boosting, from scratch    0.8174  (std 0.0197)
  XGBoost, simple           0.8213  (std 0.0198)
  XGBoost, defaults         0.7957  (std 0.0253)
  scikit-learn, defaults    0.8078  (std 0.0212)
  XGBoost simple minus forest +0.0216 [+0.0125, +0.0308]; better on 18, worse on 2 of 20
Breast cancer: 569 rows, 20 splits
  forest, from scratch      0.9596  (std 0.0128)
  boosting, from scratch    0.9485  (std 0.0187)
  XGBoost, simple           0.9564  (std 0.0174)
  XGBoost, defaults         0.9640  (std 0.0166)
  scikit-learn, defaults    0.9629  (std 0.0111)
  XGBoost simple minus forest -0.0032 [-0.0096, +0.0023]; better on 7, worse on 8 of 20
Digits: 1797 rows, 6 splits
  forest, from scratch      0.9712  (std 0.0054)
  boosting, from scratch         n/a (binary only)
  XGBoost, simple           0.9576  (std 0.0090)
  XGBoost, defaults         0.9601  (std 0.0075)
  scikit-learn, defaults    0.9697  (std 0.0028)
  XGBoost simple minus forest -0.0136 [-0.0198, -0.0074]; better on 0, worse on 6 of 6
the splits share rows, so these intervals are too narrow, a rough guide only
Actual output from this lesson's run.
```

What it means

There is no single winner. Titanic: XGBoost simple is best (0.8213), +0.0216 above the forest. Breast cancer: every model but Day 109's is within noise (forest against XGBoost -0.0032). Digits: the from-scratch forest beats XGBoost on all 6 splits (0.9712 against 0.9576), though not by much against scikit-learn (0.9697). 'Use XGBoost' is good advice about speed and tooling. As a statement about accuracy, on these tables, it is not supported. These are small datasets, where differences between good models are small.

WHY CHOOSE THIS?

Easy to include many models once each one is a function call.

WHY NOT THIS? (TRADEOFFS)

Three small tables; results on large tables can differ.

6. Explaining one prediction

DEFINITION — start here, no prior knowledge assumed

SHAP values split one prediction into a contribution from each feature, using Shapley values from game theory. XGBoost computes them exactly for trees. For every row, the contributions and the base add up to the prediction.

REAL-WORLD SCENARIO

Splitting a restaurant bill fairly when some dishes were shared: each person's share depends on who else was at the table.

MATHEMATICS

$$F(x) = base + \Sigma_{features\ j}\ \phi_j(x), \quad \phi_j = \text{feature } j\text{'s average marginal contribution over all orderings}$$

ACTUAL OUTPUT

```
Titanic with Day 108's PassengerId and coin columns, 150 rounds, test accuracy 0.7948
SHAP values plus the base add up to each prediction: largest gap 2.9e-06 log-odds
feature        mean |SHAP|
is_female           1.3211
Pclass              0.7043
Age                 0.4451
Fare                0.4257
PassengerId         0.3112
SibSp               0.1544
coin                0.1128
Parch               0.0380
one passenger: the highest predicted survival in the test set, 0.994, base log-odds -0.485
  Pclass = 1: +2.015; is_female = 1: +1.857; PassengerId = 821: +1.365; Fare = 93.5: +0.799; Age = 52: -0.536
and the lowest, 0.003:
  Fare = 6.45: -1.629; Age = 43: -1.209; PassengerId = 819: -1.045; is_female = 0: -0.990; Pclass = 3: -0.577
Actual output from this lesson's run.
```

What it means

The sum holds to 2.9e-06. Unlike Day 108's measures, SHAP answers a question about one passenger: Pclass +2.015, is_female +1.857. But note the third term for that passenger: PassengerId +1.365. A row number moved this prediction a long way, and on average it ranks above SibSp (0.3112 against 0.1544). SHAP is correct: the model really did use it. So SHAP is a faithful account of the model, and that is why it cannot be used as an account of the world. Day 108's lesson carries over: a good explanation of an overfitted model explains the overfitting.

WHY CHOOSE THIS?

Exact, additive, and works for one row, which is what a customer or a regulator asks about.

WHY NOT THIS? (TRADEOFFS)

Explains the model, not cause; correlated features share credit in ways that can mislead.

7. Saving and loading

DEFINITION — start here, no prior knowledge assumed

A model is only useful if it can leave the training script. XGBoost saves its trees as JSON, a portable text format.

REAL-WORLD SCENARIO

Writing the recipe down, so that anyone can cook it without the chef.

ACTUAL OUTPUT

```
a 200-round model saved as JSON is 148 KB; loaded back, the largest prediction gap is 0.0e+00
predicting 171 rows from a new data object takes 0.41 ms on one thread, about 2.4 microseconds per row
asking again with the same data object takes 0.04 ms: the library keeps the last answer, so timing that
    measures a lookup, not a prediction
Actual output from this lesson's run.
```

What it means

The saved model is 148 KB and predicts identically after loading (0.0e+00). Prediction takes about 2.4 microseconds per row. A first version of this timing reported 0.3 microseconds, which was too good: asking again with the same data object takes only 0.04 ms because the library keeps the last answer, so that loop timed a lookup. A number that seems too good deserves a second look.

WHY CHOOSE THIS?

A format that outlives the training code, and fast predictions.

WHY NOT THIS? (TRADEOFFS)

A saved model does not record the data, the features or the preprocessing it needs.

Interview preparation — technical questions

Q: How does XGBoost differ from plain gradient boosting?

A: It uses gradient and curvature for both splits and leaf values, with a penalty on leaves, plus engineering such as histograms.

Q: What is the leaf value?

A: -G / (H + lambda): one Newton step, shrunk by lambda.

Q: Why are histograms faster?

A: Only bin edges are considered, not every distinct value. Here 0.73 s against 15.44 s on 100,000 rows.

Q: How does XGBoost handle missing values?

A: It learns a default direction at each question. Here it was no better than a median fill.

Q: What do you tune first?

A: Learning rate with early stopping, then depth. Tuning gained +0.0117 on breast cancer and nothing on Titanic.

Interview preparation — theoretical questions

Q: Why can filling with a median keep the signal in a gap?

A: One repeated value can be isolated by a tree. Random fill removed that: 0.7158 against 0.9313.

Q: Why not always use XGBoost?

A: On small tables a forest needs less tuning, and it won on digits. A straight line won on Diabetes on Day 109.

Q: What is a SHAP value?

A: One feature's fair share of a prediction; the shares and the base add up to it exactly.

Q: Why could SHAP mislead you?

A: It explains the model. A row number got +1.365 on one passenger because the model had overfitted.

Key takeaway and what's next

KEY TAKEAWAY

XGBoost's core is about thirty lines of Newton boosting, reproduced here tree for tree. What a library adds is engineering: about 67 times the speed on one thread at 30,000 rows, plus early stopping, saving and exact SHAP values. Claims about accuracy deserve a test: on these small tables it was not always best, and its missing-value handling gave no advantage over a median that trees can read.

- Same trees: 500 of 500 on breast cancer with lambda 1 or 10; differences elsewhere are near-ties.
- 30,000 rows: 16.64 s from scratch, 4.68 s exact, 0.25 s histograms, 0.08 s on all threads.
- Missing large values, 50% gone: native 0.9348, median 0.9313, random fill 0.7158.
- Tuning: +0.0117 on breast cancer, -0.0037 on Titanic.

- Digits: forest 0.9712, XGBoost 0.9576.

Suggested watch order, if starting from zero

- 1. XGBoost Part 1 (of 4): Regression (StatQuest) — how its trees are built.
- 2. XGBoost in Python from Start to Finish (StatQuest) — using the library on a real problem.

Day 111 preview

- **Trees against neural networks.** The week's build: a head-to-head on tabular data, assembled from the exercise parts of Days 105 to 110.